

Checking a table for the search of exp in Rocq

What `code/exptable` checks on each line, in the notation of `doc/htr.md`

Paul Zimmermann's search for the hard-to-round cases of `exp` (`doc/htr.md`) reads a table with one line per subrange of inputs. This note says what a line holds, the six conditions under which the search on that line misses no hard case, and how the Rocq files in `code/exptable/` check them all with one boolean function whose answer `true` is proved to imply the six conditions. Section 7 follows the first line of the table through every step, with its numbers.

1 Proof by a checker, for a reader who does not use Rocq

Rocq is a proof assistant: it accepts a theorem only with a proof it can verify step by step. It is also a programming language, and it can run its own programs. The files here use both sides:

- `check_table` is a **program**. It reads the table and answers `true` or `false`. It uses integers only, plus one call to a function of the Interval library that returns two numbers $L \leq \exp(x) \leq U$.
- `check_tableP` is a **theorem** about that program: **if `check_table` answers `true` on a table, then every line of the table satisfies the six conditions**. It is proved once, for every possible table.
- To prove that `in_lt` satisfies the six conditions, Rocq runs the program on it, sees `true`, and applies the theorem.

So one does not need to trust the program, or to read it: the theorem is what guarantees that it tests the right thing. What one must read is the statement of the six conditions (section 4), because that is what is proved. The run itself is done by an evaluator built into Rocq (`vm_compute`, or `native_compute`, which compiles the program to machine code first).

Two words used below. The **binade** of a positive number y is the interval $[2^e, 2^{e+1})$ that contains it. An **enclosure** of y is a pair of numbers $L \leq y \leq U$, here with $U - L$ tiny compared with y .

2 The notation of `doc/htr.md`

- $\beta = 2^{64}$ is the machine word basis, ℓ the number of words of a coefficient, so the coefficients are integers modulo β^ℓ .
- A subrange is $[x_0, x_1]$ with $x_1 = x_0 + nu$, where $u = \text{ulp}(x)$ is the same for all x in $[x_0, x_1]$. The search looks at the $n + 1$ inputs $x_0 + ju$, $j = 0, \dots, n$.
- $v = \frac{1}{2} \text{ulp}(\exp x)$, the same for all x in $[x_0, x_1]$: if $2^e \leq \exp x < 2^{e+1}$ then $v = 2^{e-53}$.
- $a_i = \exp(x_0)u^i/i!$, $0 \leq i < k$: the Taylor coefficients, so that $\exp(x_0 + ju) = a_0 + a_1j + \dots + a_{k-1}j^{k-1} + r$ with $|r| \leq \exp(x_1)(nu)^k/k!$.
- A_i is an integer with $0 \leq A_i < \beta^\ell$ and $|A_i - \beta^\ell \text{frac}(a_i/v)| < 1$.
- $P(j) = A_0 + A_1j + \dots + A_{k-1}j^{k-1}$.
- m : an input is hard to round when $\exp(x)/v$ is within 2^{-m} of an integer. E is the window of the search: $x_0 + ju$ is a candidate when $P(j)$ is within E of a multiple of β^ℓ .

3 What a line of the table holds

A line of `in_lt` is x_0 (a double, in hexadecimal), then n , then k integers $B_0, \dots, B_{k-1}$ of ℓ words each, the least significant word first:

$$B_i = P(i) \bmod \beta^\ell, \quad 0 \leq i < k.$$

This is the output of step 3 of the search in `doc/htr.md`, before the differences of step 4. It is P at $0, 1, \dots, k-1$, **not** the A_i : only $B_0 = A_0$. For `in_lt`, $k = 8$, $\ell = 5$, every x_0 is in $[2^9, 2^{10})$ so $u = 2^{-43}$, and there are 67 486 lines. On line 1, $x_0 = 0x1.4678ea18f304cp+9$, $n = 7412951889$, $2^{942} \leq \exp x_0 < 2^{943}$ so $v = 2^{889}$.

4 The six conditions

For one line, the search misses no hard case when:

1. **u is constant:** x_0 and x_1 are in the same binade $[2^9, 2^{10})$;
2. **v is constant:** $\exp x_0$ and $\exp x_1$ are in the same binade $[2^e, 2^{e+1})$;
3. **(H_A):** there are $A_0, \dots, A_{k-1}$ with $0 \leq A_i < \beta^\ell$ and $|A_i - \beta^\ell \text{frac}(a_i/v)| < 1$, and the line holds $B_i = P(i) \bmod \beta^\ell$ for $i < k$;
4. **(H_T):** for $0 \leq j \leq n$,

$$|\exp(x_0 + ju)/v - (a_0 + a_1j + \dots + a_{k-1}j^{k-1})/v| \leq \rho, \quad \rho = \exp(x_1) \frac{(nu)^k}{k!v};$$

5. **(H_E):** $E \geq \beta^\ell(2^{-m} + \rho) + (1 + n + \dots + n^{k-1})$;
6. **the window is less than half the modulus:** $2E < \beta^\ell$.

Conditions 3 to 6 are the hypotheses of `hscan_exp` in `code/APaul/rocq/TaylorLink.v` (there $M = \beta^\ell$, and its a_i is a_i/v here). Conditions 1 and 2 make u and v single numbers on the line, which the other four take for granted. Condition 6 is on the search, not on a line: it is checked once.

In Rocq, the six conditions are `line_ok` and `table_ok` in `ExpCheck.v` (annex A). ℓ appears as `l`, and $x_0 = M_0u$ with M_0 the integer significand of x_0 .

5 The parameters

They are the first definitions of `ExpCheck.v`, and nothing else in the files is a number about the table:

```
Definition beta : Z := 2 ^ 64.      (* the machine word basis          *)
Definition l : Z := 5.              (* words of an A_i or a B_i      *)
Definition k : nat := 8.            (* terms of the Taylor polynomial *)
Definition m : Z := 43.             (* identical bits after the round bit *)
Definition E : Z := 0x400001 * 2 ^ 256. (* the window of htr3.c          *)
Definition xbin : Z := 9.           (* x ranges over [2^xbin, 2^(xbin+1)) *)
Definition guard : Z := 64.         (* extra bits in the exp enclosure *)
```

The value of E . E is not in the table: it is a constant of the search program. The program that reads `in_lt`, `htr3.c` (in `code/APaul/`), uses the same E for every line:

```
static uint64_t ERR = 0x400001; // ceil(2^64*(2^-43+2^-43+2^-90))
B[0][l-1] += ERR;
```

`ERR` is added to the top word of B_0 , so its E is $0x400001 \cdot 2^{256} = 2^{278} + 2^{256}$, and that is the value in `ExpCheck.v`. For comparison, `doc/htr.md` asks $E \geq \beta^\ell(2^{-43} + 2^{-43.7} + 2^{-91})$, and line 1 alone needs $E \geq 2^{277.59}$. Condition 5 says E is **at least** a bound computed from the line, so a larger E only makes it easier.

`guard` is the number of bits beyond β^ℓ and v at which `exp` is enclosed; it changes the cost, not the result.

6 How one line is checked: `check_line`

`check_line` takes M_0 , n and $B_0, \dots, B_{k-1}$ and computes with integers only, except for one call to the certified `exp` of the Interval library.

1. **Condition 1** is three integer tests: $0 \leq n$, $2^{52} \leq M_0$ and $M_0 + n < 2^{53}$.
2. **Enclosures of `exp`.** `encl` builds the point M_0u as a float of Interval and calls its `exp` at precision $\ell \cdot 64 + 53 + \text{guard}$ bits. The answer is two floats $L_0 \leq \exp x_0 \leq U_0$. The same is done at x_1 : $L_1 \leq \exp x_1 \leq U_1$.
3. **Condition 2.** e is the exponent of L_0 , so $2^e \leq L_0 \leq \exp x_0$; the test is $U_1 < 2^{e+1}$.
4. **Condition 3.** For each $i < k$, $T = \beta^\ell a_i/v$ is $\exp(x_0)$ times the power of two $\beta^\ell u^i/v$, divided by $i!$. The checker computes, at both ends L_0 and U_0 , the nearest integer R of T and the integer

part q of T/β^ℓ , and requires the same R and the same q at both ends. Then they are those of the exact T , and $A_i = R - q\beta^\ell$ satisfies $0 \leq A_i < \beta^\ell$ and $|A_i - \beta^\ell \text{frac}(a_i/v)| = |R - T| \leq \frac{1}{2}$. Last, it computes $P(i) \bmod \beta^\ell$ from these A_i and compares with B_i .

5. **Condition 4** needs no test: it is a theorem of analysis (the Taylor remainder of $\exp$, `ExpTaylor.v`).
6. **Condition 5** is one integer inequality, with $\exp x_1$ replaced by its upper bound U_1 , which only makes ρ larger.

The A_i are not in the table and cannot be read back from it: going from the B_i to the A_i divides by $i!$, which has no inverse modulo β^ℓ . So the checker recomputes them from $\exp x_0$ and compares the B_i .

`check_table` tests condition 6, then `check_line` on every line.

7 Line 1, step by step

The first line of `in_lt` is (the B_i are shown by their most significant word only; each has five):

$$x_0 = 0x1.4678ea18f304cp+9 \approx 652.94464, \quad n = 7412951889,$$

$B_0 = 0x3be38424e0f19a8a \dots$	$B_1 = 0x3be384255a1916fa \dots$
$B_2 = 0x3be384265340936b \dots$	$B_3 = 0x3be38427cc680fdb \dots$
$B_4 = 0x3be38429c58f8c4c \dots$	$B_5 = 0x3be3842c3eb708bc \dots$
$B_6 = 0x3be3842f37de852d \dots$	$B_7 = 0x3be38432b106019d \dots$

The run of section 9 checks this line the way it checks all the others: `check_raw` reads the text with `parse`, then gives the numbers to `check_line`. The numbers below are the checker's own values when it says so; the others are computed with `check_line.py` (a Python computation with exact integers, not a proof), and give the orders of magnitude.

Reading the line. `parse` finds the fields separated by spaces: the significand digits `4678ea18f304c` of x_0 and the exponent `9` (the checker requires it to be 9), then n in decimal, then 40 words `0x...`. It groups the words by five, the least significant first, into $B_0, \dots, B_7$ (section 11 states and proves this reading).

Condition 1: u is constant. The significand of x_0 is $M_0 = 0x14678ea18f304c = 5743361827745868$, so $x_0 = M_0 u$ with $u = 2^{-43}$. Then $x_1 = x_0 + nu$ has significand $M_0 + n = 5743369240697757$, that is $x_1 = 0x1.467905b67db9dp+9 \approx 652.94549$. Both significands are between 2^{52} and 2^{53} , so x_0 and x_1 are in $[2^9, 2^{10})$, where $\text{ulp}(x) = 2^{-43}$.

Enclosures of $\exp$ (checker's values). Interval's `exp` at 437 bits (β^ℓ , 320 bits, plus 53 for v , plus `guard` = 64) returns, for x_0 , two numbers $L_0 \leq \exp(x_0) \leq U_0$ with $L_0 = p_0 \cdot 2^{505}$, p_0 an integer of 438 bits, and $U_0 = L_0 + 2^{505}$: an enclosure of relative width about 2^{-437} . At x_1 it returns $L_1 = p_1 \cdot 2^{505}$, p_1 of 438 bits, and $U_1 = L_1 + 2 \cdot 2^{505}$.

Condition 2: v is constant. L_0 lies in $[2^{942}, 2^{943})$, so $2^{942} \leq \exp(x_0)$ and $e = 942$; the upper end U_1 at x_1 is below 2^{943} . So all of $\exp(x_0), \dots, \exp(x_1)$ are in $[2^{942}, 2^{943})$ and $v = 2^{942-53} = 2^{889}$. (Here $x_0 \approx 942 \ln 2$: this line starts exactly where `exp` enters that binade, $\exp(x_0) \approx 2^{942.000000}$, and $\exp(x_1) \approx 2^{942.0012}$.)

Condition 3: the A_i , then the B_i . For each i , the checker forms $T = \beta^\ell a_i/v$ at both ends of the enclosure and finds the same integer part of T/β^ℓ (q below) and the same nearest integer R of T ; then $A_i = R - q\beta^\ell$. The distance $|R - T|$ is at most $1/2$, so (H_A) holds.

i	a_i/v	q	A_i , top word
0	$9.007199255 \cdot 10^{15}$	9007199254741449	<code>0x3be38424e0f19a8a</code>
1	1024.0000	1024	<code>0x39277c70</code>

2	$5.82 \cdot 10^{-11}$	0	0x40000000
3	$2.21 \cdot 10^{-24}$	0	0x0
4	$6.27 \cdot 10^{-38}$	0	0x0
5	$1.43 \cdot 10^{-51}$	0	0x0
6	$2.70 \cdot 10^{-65}$	0	0x0
7	$4.39 \cdot 10^{-79}$	0	0x0

From the fourth on, a_i/v is below 2^{-64} , so the top word of A_i is 0 and only the lower words are nonzero. The checker then computes $P(0), \dots, P(7)$ modulo β^ℓ from these A_i and finds exactly the eight B_i of the line. Only $B_0 = P(0) = A_0$. The other A_i are small next to A_0 (the top word of A_1 is below 2^{30} , that of A_2 is 2^{30} , the others are 0), so $P(i) = A_0 + A_1 i + \dots$ changes only the low half of the top word: this is why the eight top words look alike.

Condition 4: (H_T). Nothing to compute: the Taylor theorem gives the bound, with

$$\rho = \exp(x_1) \frac{(nu)^k}{k!v} \approx 2^{-44.00}.$$

Condition 5: (H_E). The three terms of the window are $\beta^\ell 2^{-m} = 2^{277}$, $\beta^\ell \rho \approx 2^{276.00}$ and $1 + n + \dots + n^7 \approx 2^{229.51}$, a total of about $2^{277.59}$. The checker replaces $\exp(x_1)$ by $U_1 = q_1 \cdot 2^{505}$ ($q_1 = p_1 + 2$), which can only make ρ larger, and tests one integer inequality (`window_ok`). With $\beta^\ell \rho \leq q_1 n^8 2^g / 8!$ where $g = 505 + 320 - 43 \cdot 8 - (942 - 53) = -408$, and $C = 2^{277} + (1 + n + \dots + n^7)$, it tests

$$q_1 n^8 + C \cdot 8! \cdot 2^{408} \leq E \cdot 8! \cdot 2^{408},$$

which is the window condition multiplied by $8! \cdot 2^{408}$; here $E = 2^{278} + 2^{256}$.

Condition 6 is on the whole table: $2E = 2^{279} + 2^{257} < \beta^\ell = 2^{320}$.

On this line `check_line` answers `true`; the line then satisfies the six conditions, by the theorem.

8 The theorem

Theorem `check_tableP t : check_table t = true -> table_ok t`.

It is proved once, in `ExpProof.v` (annex B), from four files:

- `ExpEncl.v`: what `encl` returns does enclose `exp`, from the correctness theorem `I.exp_correct` of `Interval`;
- `ExpArith.v`: each integer test of the checker, read in the real numbers;
- `ExpTaylor.v`: the Taylor remainder, condition 4.

No lemma is admitted. The theorem rests on the axioms of Rocq's real numbers (`sig_forall_dec`, `sig_not_dec`), functional extensionality and the excluded middle (`classic`), which the libraries bring in, and on the declarations of the primitive 63-bit integers that the `Interval` library computes with.

A table is then checked by one computation: `ExpRun.v` proves `table_ok table` by `apply check_tableP` and evaluating `check_table table` to `true`.

9 The result: the whole table

All 67 486 lines of `in_lt` pass. The Rocq file `ExpAll.v` proves, for each of the 15 slices $NN = 00, \dots, 14$:

```
Lemma lines_okNN : forall s, In s ExpDataNN.lines ->
  exists M0 n B, parse s = Some (M0, n, B) /\ line_ok M0 n B.
```

In words: every line of the table, read by `parse`, is a subrange $[x_0, x_1]$ with its n and its $B_0, \dots, B_7$ that satisfies the six conditions of section 4, for the parameters of section 5, with the E of `htr3.c`. Conditions 3 to 6 are the hypotheses of `hscan_exp` in `TaylorLink.v`, which says

that the search then misses no $x_0 + ju$ with $\exp(x_0 + ju)/v$ within 2^{-m} of an integer; the two are not yet joined in one Rocq theorem. `htr3.c` tests only the top word of B_0 against $2 \cdot \text{ERR}$, which keeps every candidate of the test $B_0 \leq 2E$ of `doc/htr.md` and adds a few: it misses nothing more.

The run was made on `roquableu` (an Intel Xeon E5-2667 server, 24 threads), with `native_compute`, the 15 checks and the 15 coverage files of section 10 at once (`make -j15 slices`), then `make all-slices`; all times are measured:

	time	per slice
15 checks (<code>ExpRunNN.v</code>)		227 to 290 s
15 coverage checks (<code>ExpCoverNN.v</code>)		29 to 41 s
<code>make -j15 slices</code> , wall	5 min 14 s	
<code>make -j15 slices</code> , processor	72 min 40 s	
<code>make all-slices</code> (with <code>ExpAll.v</code>)	10.3 s	

A check of 4 500 lines is 0.050 to 0.064 s a line, the 15 workers sharing 12 cores, and each worker uses about 605 MB.

10 The lines cover the whole range

Each line is checked alone by `check_line`; that the lines, put together, leave no input out is a separate check, in `ExpCover.v`. A line covers the significands $M_0, M_0 + 1, \dots, M_0 + n$, that is the inputs $x_0 + ju$ for $j = 0, \dots, n$. The next line must start one ulp further:

$$M_{0'} = M_0 + n + 1.$$

- `contig` parses the lines of a slice (no `exp`) and tests this for each pair of consecutive lines;
- `joins` tests it between the last line of a slice and the first line of the next;
- `cover_ok` is the theorem: if every slice passes `contig`, the slices pass `joins`, and every line satisfies the six conditions, then every significand M from the first M_0 of the table to the last $M_0 + n$ lies in $[M_0, M_0 + n]$ of a line that satisfies the six conditions.

`ExpAll.v` applies it to the 15 slices:

Theorem `table_cover M` :

```
firstM ExpData00.lines <= M <= lastM ExpData14.lines ->
exists s M0 n B, In s (concat (ExpData00.lines :: rest)) /\
  parse s = Some (M0, n, B) /\ line_ok M0 n B /\ M0 <= M <= M0 + n.
```

and `range_all` states the two ends. For `in_lt` they are (proved by `range_all` in the run of section 9):

	significand	input
first M_0	5743361827745868	$x = 0x1.4678ea18f304cp+9 \approx 652.94464$
last $M_0 + n$	6243314768165359	$x = 0x1.62e42fefa39efp+9 \approx 709.78271$

That is 499952940419492 inputs. The first one is where `exp` enters the binade $[2^{942}, 2^{943})$, since $x_0 \approx 942 \ln 2$; the last one is the x_1 that `doc/htr.md` gives for the binade $[2^{1023}, 2^{1024})$, the largest double whose `exp` is finite. So the table covers every double from the one where `exp` reaches 2^{942} to the one where it overflows.

Each slice is checked in its own file `ExpCoverNN.v` with `native_compute`, in 29 to 41 s on `roquableu` (section 9); `range_all` proves the two ends above in Rocq. On a table of 30 lines with one line removed, the check fails on the slice that holds the gap.

11 Reading the text of a line: parse

The files hold the lines of `in_lt` as text, and the Rocq function `parse` (`ExpParse.v`) turns a line into its numbers (M_0, n, B) . `ExpParseSpec.v` says in plain definitions what a line means, and proves that `parse` reads exactly that. A line is the text

$$0x1.Fp+X \ N \ 0xW_0 \ 0xW_1 \ \dots \ 0xW_{k\ell-1}$$

with single spaces between the fields, where:

- F is 1 to 13 hexadecimal digits of value f , and $M_0 = 16^{13} + f \cdot 16^{13-|F|}$, so that $x_0 = (1 + f/16^{|F|})2^9 = M_0 2^{-43}$ (a double may drop the trailing zeros of F);
- X is decimal digits of value 9 (the binade of x);
- N is decimal digits of value n ;
- each W_j is hexadecimal digits of value w_j , and $B_i = w_{i\ell} + w_{i\ell+1}\beta + \dots + w_{i\ell+\ell-1}\beta^{\ell-1}$ for $i < k$: the words of B_i , least significant first.

A digit is its character: `0` to `9` for 0 to 9, `a` to `f` for 10 to 15. This is `line_text`, and the theorem is

Theorem `parse_sound s M0 n B :`

`parse s = Some (M0, n, B) -> line_text (to_list s) M0 n B.`

whatever `parse` returns is what the text says. With it, `ExpAll.v` states the final result on the text itself:

Theorem `table_cover_text M :`

`firstM ExpData00.lines <= M <= lastM ExpData14.lines ->`
`exists s M0 n B, In s (concat (ExpData00.lines :: rest)) /\`
`line_text (to_list s) M0 n B /\ line_ok M0 n B /\`
`M0 <= M <= M0 + n.`

Every double $x = M 2^{-43}$ from the first x_0 of the table to the last x_1 lies in the subrange of a line of `in_lt` whose text says x_0 , n and $B_0, \dots, B_7$, and these satisfy the six conditions. Its axioms are those of Rocq's real numbers, functional extensionality, the excluded middle, and the primitive integers and strings. It is proved on the whole table (the run of section 9).

12 Running it

`gen.py in_lt FIRST COUNT K L ExpData.v` writes lines `FIRST + 1` to `FIRST + COUNT` of `in_lt` as the Rocq list `table` (it only parses; K and L must be `k` and `l`). `make` builds the files and `ExpRun.v`.

The table is in the repository as `code/exptable/in_lt.xz`; `xz -dk in_lt.xz` gives `in_lt`, and `sha256sum -c in_lt.sha256` checks it.

The whole table is checked in slices. `mkslices.sh in_lt 15` cuts it into 15 slices of 4 500 lines. `ExpData00.v` to `ExpData14.v` hold the lines **verbatim**, as primitive strings (`genraw.py`); the Rocq function `parse` (`ExpParse.v`) reads each one into (M_0, n, B) during the evaluation. For each slice, `ExpRunNN.v` proves that `check_raw`, which parses every line and runs `check_line` on it, answers `true`, with `native_compute`. These files use `ExpCheck.v` and `ExpParse.v` only, so the proofs can change without running them again. `make -j15 slices` runs them in parallel; `make all-slices` then builds `ExpAll.v`: every line of every slice reads as a line that satisfies the six conditions (`check_rawP`), and, with the files `ExpCoverNN.v`, the lines cover the whole range (section 10).

The text form matters for the cost. Measured on 300 lines on the desktop, Rocq reads the lines as strings in 0.5 s (168 MB), but the same numbers written as decimal integers in 55 s (1.1 GB), and in hexadecimal in 28 s: building each 320-bit number as a Rocq term is what is slow. The check of the 300 lines then takes 21.5 s with `native_compute`.

Measured on the first 10 lines, on the desktop: `check_table` evaluates to `true` in 2.48 s with `vm_compute`, and in 0.75 s with `native_compute` once the code is compiled (1.50 s the first time).

That is 0.25 s and 0.075 s a line. At 0.075 s a line, the 67 486 lines would take about 1.4 h on one core (scaled, not measured). Changing one bit of a B_i , moving x_0 by one ulp, or doubling n makes `check_line` answer `false` on line 1.

The bit-flip test (`ExpMutate.v, make test`) makes sure every coefficient and x_0 is really checked. On the first three lines of `in_lt`, which pass, it changes one bit of one B_i (the lowest bit, bit 160 and the top bit 319, for each of the 8 coefficients), or moves x_0 by one ulp either way, or changes bit 0, 20 or 51 of its significand: 29 changes a line. Rocq checks by evaluation that `check_line` answers `false` on each of them, in 7.8 s on the desktop (measured).

`check_line.py` computes the same six conditions for one line in Python, with its own integer `exp` at 1200 bits. It is a reference, not a proof; it agrees with the Rocq checker on the first 10 lines.

13 What is not checked

- The search itself: that is `TaylorLink.v`, which this check feeds.

A ExpCheck.v

```
(** * Checking a table for the hard-to-round search of exp, by reflection
```

```
The search is the one of [doc/htr.md] (Paul Zimmermann), whose
notation we keep. The input range is cut into subranges
[[x0, x1]] with [x1 = x0 + n u], [u = ulp x] and [v = ulp(exp x) / 2]
constant on each. With [a_i = exp x0 u^i / i!], each [A_i] is an
integer with [0 <= A_i < beta^l] and [|A_i - beta^l frac(a_i/v)| < 1],
and [P(j) = A_0 + A_1 j + ... + A_(k-1) j^(k-1)].
```

```
A line of the table is [(x0, n, B)] with [B_i = P(i) mod beta^l] for
[i < k], the output of step 3 of the search. [line_ok] states the
six conditions under which the search on that line misses no hard
case; [check_table] tests them all with one boolean function. This
file holds the definitions only; [ExpProof.check_tableP] proves that
a [true] answer gives [table_ok]. *)
```

```
From Stdlib Require Import ZArith Reals Lia Lra List.
```

```
From Flocq Require Import Core.
```

```
From Bignums Require Import BigZ.
```

```
From Interval Require Import Xreal Basic Specific_bigint Specific_ops.
```

```
From Interval Require Import Float_full.
```

```
Import ListNotations.
```

```
(** ** Parameters
```

```
The table and the search are described here and nowhere else, in the
notation of [doc/htr.md]. *)
```

```
Open Scope Z_scope.
```

```
Definition beta : Z := 2 ^ 64.      (* the machine word basis          *)
Definition l : Z := 5.              (* words of an A_i or a B_i      *)
Definition k : nat := 8.            (* terms of the Taylor polynomial *)
Definition m : Z := 43.             (* identical bits after the round bit *)
Definition E : Z := 0x400001 * 2 ^ 256. (* the window of htr3.c          *)
Definition xbin : Z := 9.           (* x ranges over [2^xbin, 2^(xbin+1)) *)
Definition guard : Z := 64.         (* extra bits in the exp enclosure *)
```

```
(** ** Derived constants
```

```
[beta^l = 2^lbits], [u = 2^uexp], [v = 2^(e - vsh)] for [exp x] in
[[2^e, 2^(e+1))], and the precision at which [exp] is enclosed. *)
```

```
Module SFBI2 := SpecificFloat BigIntRadix2.
```

```
Module I := FloatIntervalFull SFBI2.
```

```
Definition beta_l : Z := beta ^ l.
Definition lbits : Z := Z.log2 beta * l.
Definition uexp : Z := xbin - 52.
Definition vsh : Z := 53.
Definition prec : SFBI2.precision :=
  SFBI2.PtoP (Z.to_pos (lbits + vsh + guard)).
```

```
(** ** The conditions *)
```

```

Open Scope R_scope.

Definition bp (e : Z) : R := bpow radix2 e.
Definition u : R := bp uexp.
Definition fracR (r : R) : R := r - IZR (Zfloor r).

(** [a_i = exp x0 u^i / i!]. *)
Definition a (x0 : R) (i : nat) : R := exp x0 * u ^ i / INR (fact i).

(** [f 0 + ... + f (N-1)]. *)
Fixpoint sumR (f : nat -> R) (N : nat) : R :=
  match N with 0 => 0 | S N' => sumR f N' + f N' end.

(** [P(j) = A_0 + A_1 j + ... + A_(k-1) j^(k-1)], in integers. *)
Definition Pz (A : list Z) (j : Z) : Z :=
  fold_right Z.add 0%Z (map (fun i => (nth i A 0 * j ^ Z.of_nat i)%Z) (seq 0 k)).

(** The six conditions on a line [(x0, n, B)], [x0 = M0 u]. *)
Definition line_ok (M0 n : Z) (B : list Z) : Prop :=
  let x0 := IZR M0 * u in
  let x1 := IZR (M0 + n) * u in
  exists e : Z,
  let v := bp (e - vsh) in
  let rho := exp x1 * (IZR n * u) ^ k / (INR (fact k) * v) in
  (* 1. u is constant on [x0, x1] *)
  (0 <= n)%Z /\ bp xbin <= x0 /\ x1 < bp (xbin + 1) /\
  (* 2. v is constant on [x0, x1] *)
  bp e <= exp x0 /\ exp x1 < bp (e + 1) /\
  (* 3. (H_A), and the table holds P(0) .. P(k-1) *)
  (exists A : list Z, length A = k /\ length B = k /\
    (forall i, (i < k)%nat ->
      (0 <= nth i A 0%Z < beta_l)%Z /\
      Rabs (IZR (nth i A 0%Z) - IZR beta_l * fracR (a x0 i / v)) < 1) /\
    (forall i, (i < k)%nat ->
      nth i B 0%Z = (Pz A (Z.of_nat i) mod beta_l)%Z)) /\
  (* 4. (H_T), the Taylor remainder *)
  (forall j, (0 <= j <= n)%Z ->
    Rabs (exp (x0 + IZR j * u) / v - sumR (fun i => a x0 i / v * IZR j ^ i) k)
      <= rho) /\
  (* 5. (H_E), the window *)
  IZR beta_l * (bp (- m) + rho) + sumR (fun i => IZR n ^ i) k <= IZR E.

(** Condition 6, [2 E < beta^l], is on the search, not on a line. *)
Definition table_ok (t : list (Z * Z * list Z)) : Prop :=
  (2 * E < beta_l)%Z /\
  forall M0 n B, In (M0, n, B) t -> line_ok M0 n B.

Close Scope R_scope.

(** ** The checker **)

Open Scope bool_scope.

(** The float [N u]. *)
Definition xpt (N : Z) : SFBI2.type :=

```

```

Specific_ops.Float (BigZ.of_Z N) (BigZ.of_Z uexp).

(** A positive float as its significand and exponent. *)
Definition getF (f : SFBI2.type) : option (positive * Z) :=
  match SFBI2.toF f with Basic.Float false p e => Some (p, e) | _ => None end.

(** An enclosure  $[[mL \cdot 2^{fL}, mU \cdot 2^{fU}]]$  of  $[\exp(N \cdot u)]$ . *)
Definition encl (N : Z) : option (Z * Z * Z * Z) :=
  if SFBI2.valid_lb (xpt N) && SFBI2.valid_ub (xpt N) &&
    match getF (xpt N) with
    | Some (p, e) => Z.eqb (Z.pos p) N && Z.eqb e uexp
    | None => false
  end
then
  match I.exp prec (Float.Ibnd (xpt N) (xpt N)) with
  | Float.Ibnd lo up =>
    if SFBI2.valid_lb lo && SFBI2.valid_ub up then
      match getF lo, getF up with
      | Some (pL, fL), Some (pU, fU) => Some (Z.pos pL, fL, Z.pos pU, fU)
      | _, _ => None
    end
    else None
  | Float.Inan => None
  end
else None.

(**  $\lfloor p \cdot 2^g / fk + h / 2 \rfloor$ , in integers. *)
Definition flr (p g fk h : Z) : Z :=
  if Z.leb 0 g then (2 * p * 2 ^ g + h * fk) / (2 * fk)
  else (2 * p + h * fk * 2 ^ (- g)) / (2 * fk * 2 ^ (- g)).

(**  $[p \cdot 2^f < 2^g]$ , for  $[p \geq 1]$ . *)
Definition lt2 (p f g : Z) : bool :=
  if Z.leb g f then false else Z.ltb p (2 ^ (g - f)).

(**  $[A_i]$ , from the enclosure of  $[\exp x_0]$ :  $[\beta^l a_i / v]$  is
 $[p \cdot 2^{(f + gi) / i!}]$ , its nearest integer  $[R]$  and its integer part
 $[q]$  (in units of  $[\beta^l]$ ) must be the same at both ends. *)
Definition coefA (mL fL mU fU e : Z) (i : nat) (fk : Z) : option Z :=
  let gi := lbits + uexp * Z.of_nat i - (e - vsh) in
  let R := flr mL (fL + gi) fk 1 in
  let q := flr mL (fL + gi - lbits) fk 0 in
  let Ai := R - q * beta_l in
  if Z.eqb R (flr mU (fU + gi) fk 1) &&
    Z.eqb q (flr mU (fU + gi - lbits) fk 0) &&
    Z.leb 0 Ai && Z.ltb Ai beta_l
  then Some Ai else None.

(**  $[A_i \dots A_{(i+c-1)}]$ ,  $[fk]$  being  $[i!]$ . *)
Fixpoint coefs (mL fL mU fU e : Z) (i : nat) (fk : Z) (c : nat)
: option (list Z) :=
  match c with
  | 0 => Some nil
  | S c' =>
    match coefA mL fL mU fU e i fk,
    coefs mL fL mU fU e (S i) (fk * Z.of_nat (S i)) c' with

```

```

    | Some Ai, Some As => Some (Ai :: As)
    | _, _ => None
  end
end.

(** [1 + n + ... + n^(k-1)]. *)
Definition sumn (n : Z) : Z :=
  fold_right Z.add 0 (map (fun i => n ^ Z.of_nat i) (seq 0 k)).

(** (H_E) with [exp x1 <= mU 2^fU]:
    [beta^l (2^-m + mU 2^fU (n u)^k / (k! v)) + sumn n <= E]. *)
Definition window_ok (mU fU e n : Z) : bool :=
  let X := mU * n ^ Z.of_nat k in
  let g := fU + lbits + uexp * Z.of_nat k - (e - vsh) in
  let C := 2 ^ (lbits - m) + sumn n in
  let kf := Z.of_nat (fact k) in
  if Z.leb 0 g then Z.leb (X * 2 ^ g + C * kf) (E * kf)
  else Z.leb (X + C * kf * 2 ^ (- g)) (E * kf * 2 ^ (- g)).

Definition check_line (M0 n : Z) (B : list Z) : bool :=
  Z.leb 0 n && Z.leb (2 ^ 52) M0 && Z.ltb (M0 + n) (2 ^ 53) &&
  Nat.eqb (length B) k &&
  match encl M0, encl (M0 + n) with
  | Some (mL0, fL0, mU0, fU0), Some (mL1, fL1, mU1, fU1) =>
    let e := Z.log2 mL0 + fL0 in
    lt2 mU1 fU1 (e + 1) && window_ok mU1 fU1 e n &&
    match coefs mL0 fL0 mU0 fU0 e 0 1 k with
    | Some A =>
      forallb (fun i => Z.eqb (nth i B 0) (Pz A (Z.of_nat i) mod beta_l))
        (seq 0 k)
    | None => false
  end
  | _, _ => false
end.

Definition check_table (t : list (Z * Z * list Z)) : bool :=
  Z.ltb (2 * E) beta_l && Z.leb m lbits &&
  forallb (fun r => match r with (M0, n, B) => check_line M0 n B end) t.

```

B ExpProof.v

```

(** * A [true] answer of the checker gives the six conditions *)

From Stdlib Require Import ZArith Reals Lia Lra List Bool.
From Flocq Require Import Core.
From ExpTable Require Import ExpCheck ExpParse ExpTaylor ExpArith ExpEncl.

Open Scope R_scope.

(** The parameters leave room for the [2^-m] term of the window. *)
Lemma m_le_lbits : (m <= lbits)%Z.
Proof. apply Z.leb_le; vm_compute; reflexivity. Qed.

(** [N u] is in [[2^xbin, 2^(xbin+1))] when [N] has 53 bits. *)
Lemma x_lo N : (2 ^ 52 <= N)%Z -> bp xbin <= IZR N * u.

```

```

Proof.
intros hN; unfold u, uexp, bp.
replace xbin with (52 + (xbin - 52))%Z at 1 by ring.
rewrite bpow_plus; apply Rmult_le_compat_r; [apply bpow_ge_0|].
rewrite <- IZR_Zpower by lia; apply IZR_le; exact hN.
Qed.

Lemma x_hi N : (N < 2 ^ 53)%Z -> IZR N * u < bp (xbin + 1).
Proof.
intros hN; unfold u, uexp, bp.
replace (xbin + 1)%Z with (53 + (xbin - 52))%Z by ring.
rewrite bpow_plus; apply Rmult_lt_compat_r; [apply bpow_gt_0|].
rewrite <- IZR_Zpower by lia; apply IZR_lt; exact hN.
Qed.

Theorem check_lineP M0 n B : check_line M0 n B = true -> line_ok M0 n B.
Proof.
unfold check_line.
destruct (encl M0) as [[[[mL0 fL0] mU0] fU0]] eqn:h0;
  [|rewrite !andb_false_r; discriminate].
destruct (encl (M0 + n)) as [[[[mL1 fL1] mU1] fU1]] eqn:h1;
  [|rewrite !andb_false_r; discriminate].
set (e := (Z.log2 mL0 + fL0)%Z).
destruct (coefs mL0 fL0 mU0 fU0 e 0 1 k) as [A] eqn:hc;
  [|rewrite !andb_false_r; discriminate].
intros h.
apply andb_prop in h as [h hm].
apply andb_prop in hm as [hm hB].
apply andb_prop in hm as [hlt hw].
apply andb_prop in h as [h hlen].
apply andb_prop in h as [h h53].
apply andb_prop in h as [hn h52].
apply Z.leb_le in hn; apply Z.leb_le in h52; apply Z.ltb_lt in h53.
apply Nat.eqb_eq in hlen.
destruct (encl_ok _ _ _ _ h0) as [pL0 [pU0 [lo0 up0]]].
destruct (encl_ok _ _ _ _ h1) as [pL1 [pU1 [lo1 up1]]].
exists e; cbv zeta.
assert (hx1 : IZR (M0 + n) * u = IZR M0 * u + IZR n * u)
  by (rewrite plus_IZR; ring).
repeat split.
- (* 1. u is constant *)
  exact hn.
- apply x_lo; exact h52.
- apply x_hi; exact h53.
- (* 2. v is constant *)
  apply Rle_trans with (IZR mL0 * bp fL0); [apply log2_le; exact pL0|].
  exact lo0.
- apply Rle_lt_trans with (IZR mU1 * bp fU1); [exact up1|].
  apply lt2_ok; [lia|exact hlt].
- (* 3. (H_A), and the table holds P(0) .. P(k-1) *)
  exists A.
  destruct (coefs_ok _ _ _ _ _ _ _ _ _ _ eq_refl (conj lo0 up0) hc)
    as [h1A hA].
  split; [exact h1A|]; split; [exact hlen|]; split.
+ intros i hi; destruct (hA i hi) as [hA1 hA2]; split; [exact hA1|].
  exact hA2.

```

```

+ intros i hi.
  rewrite forallb_forall in hB.
  apply Z.eqb_eq, hB, in_seq; lia.
- (* 4. (H_T) *)
  intros j hj; rewrite hx1.
  apply HT_ok; [apply bpow_gt_0|lia].
- (* 5. (H_E) *)
  apply window_okP with (mU := mU1) (fU := fU1);
    [exact m_le_lbits|exact hn| |exact hw].
  split; [apply Rlt_le, exp_pos|exact up1].
Qed.

Theorem check_tableP t : check_table t = true -> table_ok t.
Proof.
unfold check_table; intros h.
apply andb_prop in h as [h hall].
apply andb_prop in h as [h2E _].
split; [apply Z.ltb_lt; exact h2E|].
intros M0 n B hin.
rewrite forallb_forall in hall.
apply check_lineP; exact (hall _ hin).
Qed.

(** The same on the text of the lines: each one reads as a line that
    satisfies the six conditions. *)
Theorem check_rawP ls : check_raw ls = true ->
  (2 * E < beta_l)%Z /\
  forall s, In s ls -> exists M0 n B,
    parse s = Some (M0, n, B) /\ line_ok M0 n B.
Proof.
unfold check_raw; intros h.
apply andb_prop in h as [h hall].
apply andb_prop in h as [h2E _].
split; [apply Z.ltb_lt; exact h2E|].
intros s hin.
rewrite forallb_forall in hall.
specialize (hall _ hin).
destruct (parse s) as [[M0 n] B|]; [|discriminate].
exists M0, n, B; split; [reflexivity|].
apply check_lineP; exact hall.
Qed.

```

C ExpEncl.v

```

(** * The enclosure of exp given by Interval *)

From Stdlib Require Import ZArith Reals Lia Lra.
From Flocq Require Import Core.
From Interval Require Import Xreal Basic Specific_bigint Specific_ops.
From ExpTable Require Import ExpCheck.

Open Scope R_scope.

(* A positive float with a radix-2 exponent is p 2^e. *)

```

```

Lemma FtoR_pos p e : FtoR radix2 false p e = IZR (Z.pos p) * bp e.
Proof.
unfold bp; destruct e as [|q|q|]; cbv beta iota zeta delta [FtoR bpow].
- lra.
- rewrite mult_IZR; reflexivity.
- reflexivity.
Qed.

(* The value of a float read by getF. *)
Lemma getF_val f p e : getF f = Some (p, e) ->
  SFBI2.toX f = Xreal (IZR (Z.pos p) * bp e).
Proof.
unfold getF, SFBI2.toX.
destruct (SFBI2.toF f) as [| [|] p' e']; try discriminate.
intros H; injection H; intros <- <-; simpl FtoX; rewrite <- FtoR_pos.
reflexivity.
Qed.

Lemma encl_ok N mL fL mU fU : encl N = Some (mL, fL, mU, fU) ->
  (0 < mL)%Z /\ (0 < mU)%Z /\
  IZR mL * bp fL <= exp (IZR N * u) <= IZR mU * bp fU.
Proof.
unfold encl.
destruct (SFBI2.valid_lb (xpt N)) eqn:VL; [|discriminate|.
destruct (SFBI2.valid_ub (xpt N)) eqn:VU; [|discriminate|.
destruct (getF (xpt N)) as [|p e|] eqn:GX; [|discriminate|.
destruct (Z.eqb_spec (Z.pos p) N) as [HN|]; [|discriminate|.
destruct (Z.eqb_spec e uexp) as [He|]; [|discriminate|.
simpl andb; cbv iota.
(* The point interval contains N u. *)
assert (Hc : Interval.contains
  (I.convert (Float.Ibnd (xpt N) (xpt N))) (Xreal (IZR N * u))).
{ unfold I.convert; change I.F.valid_lb with SFBI2.valid_lb.
  change I.F.valid_ub with SFBI2.valid_ub; rewrite VL, VU.
  change I.F.toX with SFBI2.toX; rewrite (getF_val _ _ _ GX), HN, He.
  unfold u; simpl; lra. }
generalize (I.exp_correct prec _ _ Hc).
destruct (I.exp prec (Float.Ibnd (xpt N) (xpt N))) as [|lo up|];
[discriminate|].
unfold I.convert.
change I.F.valid_lb with SFBI2.valid_lb.
change I.F.valid_ub with SFBI2.valid_ub.
change (SFBI2.valid_lb lo) with true; rewrite Bool.andb_true_l.
destruct (SFBI2.valid_ub up); [|intros _ H; discriminate H|.
destruct (getF lo) as [|pL fL'|] eqn:GL; [|intros _ H; discriminate H|.
destruct (getF up) as [|pU fU'|] eqn:GU; [|intros _ H; discriminate H|.
change I.F.toX with SFBI2.toX.
rewrite (getF_val _ _ _ GL), (getF_val _ _ _ GU).
intros [H1 H2] H; injection H; intros <- <- <- <-.
split; [lia|split; [lia|]].
split; assumption.
Qed.

```

D ExpArith.v

```
(** * The integer tests of the checker, read in the reals *)

From Stdlib Require Import ZArith Reals Lia Lra List.
From Flocq Require Import Core.
From ExpTable Require Import ExpCheck.

Open Scope R_scope.

(** [flr] is the floor it is named after. *)
Lemma flrE p g fk h : (0 < fk)%Z -> (0 <= h)%Z ->
  flr p g fk h = Zfloor (IZR p * bp g / IZR fk + IZR h / 2).
Proof.
  intros Hfk Hh; unfold flr, bp.
  assert (Hfk' : IZR fk <> 0) by (apply not_0_IZR; lia).
  destruct (Z.leb_spec 0 g) as [Hg | Hg].
  - rewrite <- Zfloor_div by lia; f_equal.
    rewrite <- (IZR_Zpower radix2 g) by lia; simpl radix_val.
    rewrite !mult_IZR, plus_IZR, !mult_IZR; field; auto.
  - rewrite <- Zfloor_div.
    2: { apply Z.neq_mul_0; split; [lia]; apply Z.pow_nonzero; lia. }
    f_equal.
    (* a negative power of two is the inverse of an integer *)
    assert (Hq : bpow radix2 g = / IZR (2 ^ (- g))).
    { rewrite (IZR_Zpower radix2) by lia; rewrite bpow_opp, Rinv_inv; easy. }
    assert (H2 : IZR (2 ^ (- g)) <> 0).
    { apply not_0_IZR; apply Z.pow_nonzero; lia. }
    rewrite Hq, !mult_IZR, plus_IZR, !mult_IZR; field; auto.
Qed.

Lemma lt2_ok p f g : (1 <= p)%Z -> lt2 p f g = true -> IZR p * bp f < bp g.
Proof.
  intros Hp; unfold lt2, bp.
  destruct (Z.leb_spec g f) as [Hgf | Hgf]; [discriminate[]].
  intros Hlt; apply Z.ltb_lt in Hlt.
  replace (bpow radix2 g) with (bpow radix2 ((g - f) + f)) by (f_equal; ring).
  rewrite bpow_plus; apply Rmult_lt_compat_r; [apply bpow_gt_0[]].
  rewrite <- (IZR_Zpower radix2) by lia; apply IZR_lt; exact Hlt.
Qed.

Lemma log2_le p f : (0 < p)%Z -> bp (Z.log2 p + f) <= IZR p * bp f.
Proof.
  intros Hp; unfold bp; rewrite bpow_plus.
  apply Rmult_le_compat_r; [apply Rlt_le, bpow_gt_0[]].
  rewrite <- (IZR_Zpower radix2) by (apply Z.log2_nonneg).
  apply IZR_le; destruct (Z.log2_spec p Hp) as [H _]; exact H.
Qed.

(** [beta^l] is the power of two of exponent [lbits]. *)
Lemma beta_lE : beta_l = (2 ^ lbits)%Z.
Proof. vm_compute; reflexivity. Qed.

Lemma lbits_ge0 : (0 <= lbits)%Z.
Proof. vm_compute; discriminate. Qed.
```

```

Lemma beta_lR : IZR beta_l = bp lbits.
Proof.
rewrite beta_lE; unfold bp; rewrite (IZR_Zpower radix2); [easy|].
exact lbits_ge0.
Qed.

(** A power of a power of two. *)
Lemma bp_pow a i : bp a ^ i = bp (a * Z.of_nat i).
Proof.
induction i as [|i IH]; [now rewrite Z.mul_0_r|].
rewrite Nat2Z.inj_succ, <- Z.add_1_r, Z.mul_add_distr_l, Z.mul_1_r.
unfold bp in *; rewrite bpow_plus, <- IH; simpl; ring.
Qed.

(** Moving the start of an integer sum. *)
Lemma fold_addE (s : list Z) a :
  fold_right Z.add a s = (fold_right Z.add 0 s + a)%Z.
Proof. induction s as [|x s IH]; simpl; lia. Qed.

(** [sumR] of powers of an integer, in integers. *)
Lemma sumR_powE n N : sumR (fun i => IZR n ^ i) N =
  IZR (fold_right Z.add 0 (map (fun i => n ^ Z.of_nat i) (seq 0 N)))%Z.
Proof.
induction N as [|N IH]; [easy|].
rewrite seq_S, map_app, fold_right_app; simpl.
rewrite fold_addE, plus_IZR, <- IH, Z.add_0_r, pow_IZR; easy.
Qed.

(** A power of two, as an integer. *)
Lemma bp_IZR g : (0 <= g)%Z -> bp g = IZR (2 ^ g).
Proof.
intros Hg; unfold bp; rewrite <- (IZR_Zpower radix2) by exact Hg; easy.
Qed.

(** A negative power of two, as the inverse of an integer. *)
Lemma bp_IZRN g : (g < 0)%Z -> bp g = / IZR (2 ^ (- g)).
Proof.
intros Hg; unfold bp; rewrite (IZR_Zpower radix2) by lia.
rewrite bpow_opp, Rinv_inv; easy.
Qed.

(** [window_ok] in the reals, before the bound on [exp x1]. *)
Lemma window_okR mU fU e n : window_ok mU fU e n = true ->
  IZR (2 ^ (lbits - m) + sumn n) + IZR (mU * n ^ Z.of_nat k) *
  bp (fU + lbits + uexp * Z.of_nat k - (e - vsh)) / IZR (Z.of_nat (fact k))
  <= IZR E.
Proof.
unfold window_ok; cbv zeta.
set (X := (mU * n ^ Z.of_nat k)%Z).
set (g := (fU + lbits + uexp * Z.of_nat k - (e - vsh))%Z).
set (C := (2 ^ (lbits - m) + sumn n)%Z).
set (kf := Z.of_nat (fact k)).
assert (Hkf : (0 < kf)%Z) by (pose proof (lt_0_fact k); lia).
assert (HkfR : 0 < IZR kf) by (apply IZR_lt; exact Hkf).
clearbody X g C kf; generalize E; intros EE.
destruct (Z.leb_spec 0 g) as [Hg | Hg]; intros H; apply Z.leb_le, IZR_le in H;

```

```

rewrite ?plus_IZR, ?mult_IZR, ?plus_IZR, ?mult_IZR in H.
- rewrite bp_IZR by exact Hg.
  (* field and lra only see real variables, or the kernel check is slow *)
  generalize (IZR X) (IZR C) (IZR (2 ^ g)) (IZR EE) H.
  generalize (IZR kf) HkfR; clear; intros kr Hk xr cr pr er H.
  apply Rmult_le_reg_r with (1 := Hk).
  replace ((cr + xr * pr / kr) * kr) with (xr * pr + cr * kr) by (field; lra).
  exact H.
- rewrite bp_IZRN by exact Hg.
  assert (HP : 0 < IZR (2 ^ (- g))).
  { apply IZR_lt; apply Z.pow_pos_nonneg; lia. }
  generalize (IZR X) (IZR C) (IZR (2 ^ (- g))) (IZR EE) H HP.
  generalize (IZR kf) HkfR; clear; intros kr Hk xr cr pr er H Hp.
  apply Rmult_le_reg_r with (r := kr * pr); [now apply Rmult_lt_0_compat|].
  replace ((cr + xr * / pr / kr) * (kr * pr))
    with (xr + cr * kr * pr) by (field; lra).
  lra.
Qed.

(** (H_E), from an upper bound [mU 2^fU] of [y = exp x1]. *)
Lemma window_okP mU fU e n y :
  (m <= lbits)%Z -> (0 <= n)%Z -> 0 <= y <= IZR mU * bp fU ->
  window_ok mU fU e n = true ->
  IZR beta_l * (bp (- m) + y * (IZR n * u) ^ k / (INR (fact k) * bp (e - vsh)))
  + sumR (fun i => IZR n ^ i) k <= IZR E.
Proof.
intros Hm Hn Hy Hw.
eapply Rle_trans; [| exact (window_okR _ _ _ Hw)].
rewrite sumR_powE; fold (sumn n).
rewrite plus_IZR, mult_IZR, <- pow_IZR, <- INR_IZR_INZ.
rewrite <- bp_IZR by lia.
rewrite beta_lR.
unfold u; rewrite Rpow_mult_distr, bp_pow.
replace (bp (lbits - m)) with (bp lbits * bp (- m))
  by (unfold bp; rewrite <- bpow_plus; f_equal; ring).
replace (bp (fU + lbits + uexp * Z.of_nat k - (e - vsh)))
  with (bp fU * bp lbits * bp (uexp * Z.of_nat k) / bp (e - vsh))
  by (unfold bp, Rdiv; rewrite <- bpow_opp, <- !bpow_plus; f_equal; ring).
assert (HB : 0 < bp lbits) by apply bpow_gt_0.
assert (HU : 0 < bp (uexp * Z.of_nat k)) by apply bpow_gt_0.
assert (HV : 0 < bp (e - vsh)) by apply bpow_gt_0.
assert (HK : 0 < INR (fact k)) by apply INR_fact_lt_0.
assert (HN : 0 <= IZR n ^ k) by (apply pow_le, IZR_le; exact Hn).
(* only real variables are left *)
revert Hy HB HU HV HK HN.
generalize (bp lbits) (bp (- m)) (bp (uexp * Z.of_nat k)) (bp (e - vsh)).
generalize (INR (fact k)) (IZR n ^ k) (IZR (sumn n)) (IZR mU) (bp fU).
clear; intros K N S F1 F2 B M U V Hy HB HU HV HK HN.
assert (Hc : 0 <= N * U * B / (K * V)).
{ unfold Rdiv; apply Rmult_le_pos.
  - apply Rmult_le_pos; [apply Rmult_le_pos|]; lra.
  - apply Rlt_le, Rinv_0_lt_compat, Rmult_lt_0_compat; easy. }
replace (B * (M + y * (N * U) / (K * V)) + S)
  with (B * M + S + y * (N * U * B / (K * V))) by (field; lra).
replace (B * M + S + F1 * N * (F2 * B * U / V) / K)
  with (B * M + S + (F1 * F2) * (N * U * B / (K * V))) by (field; lra).

```

```

apply Rplus_le_compat_l, Rmult_le_compat_r; [exact Hc | apply Hy].
Qed.

(** A floor that is the same at both ends is the same in between. *)
Lemma Zfloor_mid a b d : a <= b <= d -> Zfloor a = Zfloor d ->
  Zfloor b = Zfloor a.
Proof.
intros [Hab Hbd] Had.
apply Zfloor_le in Hab; apply Zfloor_le in Hbd; lia.
Qed.

(** (H_A) for one coefficient, from an enclosure of [y = exp x0]. *)
Lemma coefA_ok mL fL mU fU e i fk Ai y :
  fk = Z.of_nat (fact i) ->
  IZR mL * bp fL <= y <= IZR mU * bp fU ->
  coefA mL fL mU fU e i fk = Some Ai ->
  (0 <= Ai < beta_l)%Z /\
  Rabs (IZR Ai - IZR beta_l * fracR (y * u ^ i / INR (fact i) / bp (e - vsh)))
  < 1.
Proof.
intros Hfk Hy H.
assert (Hfk0 : (0 < fk)%Z) by (pose proof (lt_0_fact i); lia).
unfold coefA in H; cbv zeta in H.
set (gi := (lbits + uexp * Z.of_nat i - (e - vsh))%Z) in H.
set (r := flr mL (fL + gi) fk 1) in H.
set (q := flr mL (fL + gi - lbits) fk 0) in H.
destruct (&&_) eqn:Hb; [|discriminate].
injection H as <-.
rewrite !Bool.andb_true_iff, Z.eqb_eq, Z.eqb_eq, Z.leb_le, Z.ltb_lt in Hb.
destruct Hb as [[H1 H2] H3] H4].
split; [lia|].
unfold r, q in *.
rewrite !flrE in H1, H2 by lia.
rewrite !flrE by lia.
clear H3 H4.
assert (HfkR : 0 < IZR fk) by (apply IZR_lt; exact Hfk0).
assert (HB : 0 < bp lbits) by apply bpow_gt_0.
(* the argument of fracR is y (2^gi / i!) / beta^l *)
assert (Hc : y * u ^ i / INR (fact i) / bp (e - vsh) =
  y * (bp gi / IZR fk) / bp lbits).
{ rewrite INR_IZR_INZ, <- Hfk; unfold u; rewrite bp_pow.
  unfold gi; replace (lbits + uexp * Z.of_nat i - (e - vsh))%Z
    with (lbits + uexp * Z.of_nat i + - (e - vsh))%Z by ring.
  assert (HV : 0 < bp (e - vsh)) by apply bpow_gt_0.
  unfold bp in *; rewrite !bpow_plus, bpow_opp; field; lra. }
rewrite Hc; clear Hc; clearbody gi.
assert (Ha : forall z f, IZR z * bp (f + gi) / IZR fk =
  IZR z * bp f * (bp gi / IZR fk)).
{ intros z f; unfold bp; rewrite bpow_plus; field; lra. }
assert (Hb : forall z f, IZR z * bp (f + gi - lbits) / IZR fk + 0 / 2 =
  IZR z * bp f * (bp gi / IZR fk) / bp lbits).
{ intros z f; replace (f + gi - lbits)%Z with (f + gi + - lbits)%Z by ring.
  unfold bp; rewrite !bpow_plus, bpow_opp.
  unfold bp in HB; field; lra. }
rewrite !Ha, !Hb in *; clear Ha Hb.
assert (Hc : 0 < bp gi / IZR fk).

```

```

{ apply Rdiv_lt_0_compat; [apply bpow_gt_0 | exact HfkR]. }
rewrite minus_IZR, mult_IZR, beta_lR.
unfold fracR.
(* only real variables are left *)
revert Hy H1 H2 HB Hc.
generalize (IZR mL * bp fL) (IZR mU * bp fU) (bp gi / IZR fk) (bp lbits).
clear; intros lo hi c B Hy H1 H2 HB Hc.
assert (Hlo : lo * c <= y * c) by (apply Rmult_le_compat_r; lra).
assert (Hhi : y * c <= hi * c) by (apply Rmult_le_compat_r; lra).
rewrite <- (Zfloor_mid (lo * c / B) (y * c / B) (hi * c / B));
[| | exact H2].
2: { unfold Rdiv; split; apply Rmult_le_compat_r; try lra;
    apply Rlt_le, Rinv_0_lt_compat; exact HB. }
rewrite <- (Zfloor_mid (lo * c + 1 / 2) (y * c + 1 / 2) (hi * c + 1 / 2));
[| lra | exact H1].
pose proof (Zfloor_lb (y * c + 1 / 2)).
pose proof (Zfloor_ub (y * c + 1 / 2)).
replace (B * (y * c / B - IZR (Zfloor (y * c / B))))
  with (y * c - IZR (Zfloor (y * c / B)) * B) by (field; lra).
apply Rabs_def1; lra.
Qed.

(** (H_A) for the [c] coefficients from the [i]-th on. *)
Lemma coefs_ok mL fL mU fU e y c i fk As :
  fk = Z.of_nat (fact i) ->
  IZR mL * bp fL <= y <= IZR mU * bp fU ->
  coefs mL fL mU fU e i fk c = Some As ->
  length As = c /\
  forall j, (j < c)%nat ->
    (0 <= nth j As 0%Z < beta_l)%Z /\
    Rabs (IZR (nth j As 0%Z) - IZR beta_l *
      fracR (y * u ^ (i + j) / INR (fact (i + j)) / bp (e - vsh))) < 1.
Proof.
intros Hfk Hy; revert i fk As Hfk.
induction c as [|c IH]; intros i fk As Hfk H; cbn [coefs] in H.
- injection H as <-; split; [easy | intros j Hj; lia].
- destruct (coefA mL fL mU fU e i fk) as [Ai] eqn:HA; [|discriminate].
  destruct (coefs mL fL mU fU e (S i) (fk * Z.of_nat (S i)) c) as [As']
    eqn:HAs; [|discriminate].
  injection H as <-.
  (* the next factorial *)
  assert (Hfk' : (fk * Z.of_nat (S i))%Z = Z.of_nat (fact (S i))).
  { change (fact (S i)) with (S i * fact i)%nat.
    rewrite Nat2Z.inj_mul, Hfk; ring. }
  destruct (IH _ _ Hfk' HAs) as [Hl HAs'].
  split; [simpl; lia].
  intros [|j] Hj.
  + rewrite Nat.add_0_r; exact (coefA_ok _ _ _ _ _ _ _ Hfk Hy HA).
  + replace (i + S j)%nat with (S i + j)%nat by lia.
    apply HAs'; lia.
Qed.

```

E ExpTaylor.v

```

(** * The Taylor remainder of exp: condition 4, (H_T) *)

From Stdlib Require Import ZArith Reals Lia Lra.
From Coquelicot Require Import Coquelicot.
From ExpTable Require Import ExpCheck.

Open Scope R_scope.

(* Every derivative of exp is exp. *)
Lemma Derive_n_exp (m : nat) (t : R) : Derive_n exp m t = exp t.
Proof.
  revert t; induction m as [|m IH]; intros t; [reflexivity|].
  simpl; rewrite (Derive_ext _ _ _ IH).
  now apply is_derive_unique, is_derive_exp.
Qed.

(* exp is differentiable [m] times everywhere. *)
Lemma ex_derive_n_exp (m : nat) (t : R) : ex_derive_n exp m t.
Proof.
  destruct m as [|m]; [exact I|simpl].
  apply (ex_derive_ext exp); [intros s; now rewrite Derive_n_exp|].
  eexists; apply is_derive_exp.
Qed.

(* [sumR] with [N + 1] terms is the Stdlib [sum_f_R0] up to [N]. *)
Lemma sumR_sum_f (f : nat -> R) (N : nat) : sumR f (S N) = sum_f_R0 f N.
Proof.
  induction N as [|N IH]; [simpl; ring|].
  change (sumR f (S N) + f (S N) = sum_f_R0 f N + f (S N)); now rewrite IH.
Qed.

(** For [h >= 0], the Taylor polynomial of degree [N-1] of [exp] at [x0]
    is below [exp (x0 + h)] by at most [exp (x0 + h) h^N / N!]. *)
Lemma taylor_exp (x0 h : R) (N : nat) : 0 <= h ->
  Rabs (exp (x0 + h) - sumR (fun i => exp x0 * h ^ i / INR (fact i)) N)
  <= exp (x0 + h) * h ^ N / INR (fact N).
Proof.
  intros Hh.
  destruct N as [|N].
  simpl; rewrite Rminus_0_r, Rabs_right by (left; apply exp_pos); lra.
  destruct Hh as [Hh|<-].
  2:{ (* At [h = 0] the sum is [exp x0] and the bound is [0]. *)
    rewrite Rplus_0_r, sumR_sum_f, pow_i by lia.
    replace (sum_f_R0 _ N) with (exp x0).
    rewrite Rminus_diag, Rabs_R0, Rmult_0_r; unfold Rdiv; lra.
    induction N as [|N IH]; [simpl; field|].
    rewrite tech5, <- IH, pow_i by lia; unfold Rdiv; ring. }
  (* Lagrange: the remainder is [h^(N+1) / (N+1)! exp z], [z] in the range. *)
  destruct (Taylor_Lagrange exp N x0 (x0 + h)) as [z [Hz Hz']]; [lra|].
  intros t _ m _; apply ex_derive_n_exp.
  rewrite sumR_sum_f.
  replace (sum_f_R0 _ N) with
    (sum_f_R0 (fun m => (x0 + h - x0) ^ m / INR (fact m) * Derive_n exp m x0)
      N).

```

```

2:{ apply sum_eq; intros i _; rewrite Derive_n_exp.
  replace (x0 + h - x0) with h by ring; unfold Rdiv; ring. }
rewrite Hz' at 1; rewrite Derive_n_exp.
replace (x0 + h - x0) with h by ring.
assert (0 < INR (fact (S N))) by apply INR_fact_lt_0.
assert (0 < h ^ S N) by (apply pow_lt; lra).
rewrite Rplus_minus_l, Rabs_right.
2:{ apply Rle_ge; unfold Rdiv; apply Rmult_le_pos; [|left; apply exp_pos|.
  apply Rmult_le_pos; [lra|left; apply Rinv_0_lt_compat; lra]. }
rewrite (Rmult_comm (exp _)); unfold Rdiv.
assert (exp z < exp (x0 + h)) by (apply exp_increasing; lra).
assert (0 < / INR (fact (S N))) by (apply Rinv_0_lt_compat; lra).
assert (0 < h ^ S N * / INR (fact (S N))) by (apply Rmult_lt_0_compat; lra).
nra.
Qed.

(* Two sums with equal terms are equal. *)
Lemma sumR_ext (f g : nat -> R) (N : nat) :
  (forall i, f i = g i) -> sumR f N = sumR g N.
Proof.
intros Hfg; induction N as [|N IH]; simpl; [easy|now rewrite IH, Hfg].
Qed.

(* A common divisor comes out of the sum. *)
Lemma sumR_div (f : nat -> R) (v : R) (N : nat) :
  sumR (fun i => f i / v) N = sumR f N / v.
Proof.
induction N as [|N IH]; simpl; [|rewrite IH]; unfold Rdiv; ring.
Qed.

(** Condition 4 of [line_ok], for [j] in [[0, n]]. *)
Lemma HT_ok (x0 v : R) (n j : Z) : 0 < v -> (0 <= j <= n)%Z ->
  Rabs (exp (x0 + IZR j * u) / v - sumR (fun i => a x0 i / v * IZR j ^ i) k)
  <= exp (x0 + IZR n * u) * (IZR n * u) ^ k / (INR (fact k) * v).
Proof.
intros Hv [Hj HjN].
assert (Hu : 0 < u) by apply Flocq.Core.Raux.bpow_gt_0.
apply IZR_le in Hj; apply IZR_le in HjN.
assert (Hh : 0 <= IZR j * u) by (apply Rmult_le_pos; lra).
(* The sum is the Taylor sum at [h = j u], divided by [v]. *)
rewrite (sumR_ext _ (fun i => exp x0 * (IZR j * u) ^ i / INR (fact i) / v)).
2:{ intros i; unfold a; rewrite Rpow_mult_distr; field.
  split; [apply not_0_INR, fact_neq_0|lra]. }
rewrite (sumR_div (fun i => exp x0 * (IZR j * u) ^ i / INR (fact i))).
set (Sj := sumR _ k).
replace (exp (x0 + IZR j * u) / v - Sj / v) with
  ((exp (x0 + IZR j * u) - Sj) / v) by (field; lra).
rewrite Rabs_div, (Rabs_right v) by lra.
assert (0 < INR (fact k)) by apply INR_fact_lt_0.
replace (_ / (INR (fact k) * v)) with
  (exp (x0 + IZR n * u) * (IZR n * u) ^ k / INR (fact k) / v)
  by (field; lra).
apply Rmult_le_compat_r; [left; apply Rinv_0_lt_compat; lra|].
eapply Rle_trans; [apply taylor_exp; lra|].
(* The bound grows with [j], so [j = n] is the worst case. *)
unfold Rdiv; apply Rmult_le_compat_r; [left; apply Rinv_0_lt_compat; lra|].

```

```

apply Rmult_le_compat.
- left; apply exp_pos.
- now apply pow_le.
- destruct (Rle_lt_or_eq_dec _ _ Hjn) as [Hlt|Heq]; [|rewrite Heq; lra].
  left; apply exp_increasing; nra.
- apply pow_incr; split; [lra]; nra.
Qed.

```

F ExpCover.v

```

(** * The lines of the table cover a whole range of inputs

  A line [(M0, n, B)] covers the significands [M0 .. M0 + n], that is
  the inputs [x0 + j u] for [j = 0 .. n]. [contig] checks, on the
  text, that each line starts one [u] after the end of the previous
  one, so that the lines together cover every significand from the
  first [M0] to the last [M0 + n]. Only [parse] is evaluated, never
  [exp]. [joins] lets the table be checked in slices. *)

From Stdlib Require Import ZArith Lia List Bool PrimString.
From ExpTable Require Import ExpCheck ExpParse.
Import ListNotations.

Open Scope Z_scope.

(** ** The check *)

(** The first significand of the first line. *)
Definition firstM (ls : list string) : Z :=
  match ls with
  | s :: _ => match parse s with Some (M0, _, _) => M0 | None => 0 end
  | [] => 0
  end.

(** The last significand of the last line. *)
Fixpoint lastM (ls : list string) : Z :=
  match ls with
  | [] => 0
  | s :: ls' =>
    match ls' with
    | [] => match parse s with Some (M0, n, _) => M0 + n | None => 0 end
    | _ => lastM ls'
    end
  end.

(** Every line parses, the first starts at [nxt], each next one starts
  right after the end of the previous one. *)
Fixpoint chain (nxt : Z) (ls : list string) : bool :=
  match ls with
  | [] => true
  | s :: ls' =>
    match parse s with
    | Some (M0, n, _) => Z.eqb M0 nxt && chain (M0 + n + 1) ls'
    | None => false
    end
  end

```

```

end.

(** The lines are not empty and follow each other. *)
Definition contig (ls : list string) : bool :=
  match ls with [] => false | _ => chain (firstM ls) ls end.

(** Each list starts right after the end of the previous one. *)
Fixpoint joins (Ls : list (list string)) : bool :=
  match Ls with
  | [] => true
  | l1 :: Ls' =>
    match Ls' with
    | [] => true
    | l2 :: _ => Z.eqb (firstM l2) (lastM l1 + 1) && joins Ls'
    end
  end.

(** Every line of [ls] reads as a line that satisfies the six
conditions. *)
Definition lines_good (ls : list string) : Prop :=
  forall s, In s ls -> exists M0 n B,
    parse s = Some (M0, n, B) /\ line_ok M0 n B.

(** ** Coverage of one list *)

Lemma chain_cover nxt ls M : chain nxt ls = true -> ls <> [] ->
  nxt <= M <= lastM ls ->
  exists s M0 n B, In s ls /\ parse s = Some (M0, n, B) /\
  M0 <= M <= M0 + n.
Proof.
revert nxt; induction ls as [|s ls IH]; intros nxt h hne hM;
  [congruence|].
cbn [chain] in h.
destruct (parse s) as [[M0 n B]] eqn:hs; [discriminate|.
apply andb_prop in h as [h1 h2]; apply Z.eqb_eq in h1; subst nxt.
destruct (Z.le_gt_dec M (M0 + n)) as [hle|hgt].
- exists s, M0, n, B; split; [left; reflexivity|split; [exact hs|lia]].
- destruct ls as [|s' ls'].
  + cbn [lastM] in hM; rewrite hs in hM; lia.
  + change (lastM (s :: s' :: ls')) with (lastM (s' :: ls')) in hM.
    destruct (IH (M0 + n + 1) h2 ltac:(discriminate) ltac:(lia))
      as [t [M1 [n1 [B1 [ht [hp hr]]]]]].
    exists t, M1, n1, B1; split; [right; exact ht|split; [exact hp|exact hr]].
Qed.

Theorem contig_cover ls M : contig ls = true ->
  firstM ls <= M <= lastM ls ->
  exists s M0 n B, In s ls /\ parse s = Some (M0, n, B) /\
  M0 <= M <= M0 + n.
Proof.
destruct ls as [|s ls]; [discriminate|].
intros h hM.
apply (chain_cover (firstM (s :: ls))); [exact h|discriminate|exact hM].
Qed.

(** ** Joining two lists *)

```

```

Lemma chain_app nxt l1 l2 : chain nxt l1 = true -> l1 <> [] ->
  chain (lastM l1 + 1) l2 = true -> chain nxt (l1 ++ l2) = true.
Proof.
revert nxt; induction l1 as [|s l1 IH]; intros nxt h hne h2;
  [congruence|].
cbn [chain app] in h |- *.
destruct (parse s) as [[M0 n] B]] eqn:hs; [|discriminate|.
apply andb_prop in h as [h1 h]; rewrite h1; cbn [andb].
destruct l1 as [|s' l1].
- cbn [lastM] in h2; rewrite hs in h2; exact h2.
- apply IH; [exact h|discriminate|exact h2].
Qed.

Lemma lastM_app l1 l2 : l2 <> [] -> lastM (l1 ++ l2) = lastM l2.
Proof.
intros hne; induction l1 as [|s l1 IH]; [reflexivity|].
destruct l1 as [|s' l1]; [|exact IH|.
destruct l2; [congruence|reflexivity|.
Qed.

Lemma contig_nil ls : contig ls = true -> ls <> [].
Proof. destruct ls; [discriminate|intros _; discriminate]. Qed.

Theorem contig_app l1 l2 : contig l1 = true -> contig l2 = true ->
  firstM l2 = lastM l1 + 1 ->
  contig (l1 ++ l2) = true /\ firstM (l1 ++ l2) = firstM l1 /\
  lastM (l1 ++ l2) = lastM l2.
Proof.
intros h1 h2 hj.
assert (hf : firstM (l1 ++ l2) = firstM l1)
  by (destruct l1; [discriminate|reflexivity]).
split; [|split; [exact hf|apply lastM_app, contig_nil, h2]].
destruct l1 as [|s l1]; [discriminate|.
change (chain (firstM ((s :: l1) ++ l2)) ((s :: l1) ++ l2) = true).
rewrite hf.
apply chain_app; [exact h1|discriminate|.
destruct l2 as [|s' l2]; [discriminate|.
rewrite <- hj; exact h2.
Qed.

(** ** Joining many lists *)

Lemma last_indep (l1 : list string) Ls d d' :
  last (l1 :: Ls) d = last (l1 :: Ls) d'.
Proof.
revert l1; induction Ls as [|l2 Ls IH]; intros l1; [reflexivity|.
exact (IH l2).
Qed.

Lemma last_cons (l0 l1 : list string) Ls :
  last (l1 :: Ls) l0 = last Ls l1.
Proof.
destruct Ls as [|l2 Ls]; [reflexivity|]; cbn [last]; apply last_indep.
Qed.

```

```

Theorem contig_concat l0 Ls :
  Forall (fun l => contig l = true) (l0 :: Ls) ->
  joins (l0 :: Ls) = true ->
  contig (concat (l0 :: Ls)) = true /\
  firstM (concat (l0 :: Ls)) = firstM l0 /\
  lastM (concat (l0 :: Ls)) = lastM (last Ls l0).
Proof.
revert l0; induction Ls as [|l1 Ls IH]; intros l0 hc hj.
- cbn [concat]; rewrite app_nil_r.
  inversion hc; subst; split; [assumption|split; reflexivity].
- inversion hc as [|x y h0 hc1]; subst.
  cbn [joins] in hj; apply andb_prop in hj as [hb hj].
  apply Z.eqb_eq in hb.
  destruct (IH l1 hc1 hj) as [hc2 [hf2 hl2]].
  change (concat (l0 :: l1 :: Ls)) with (l0 ++ concat (l1 :: Ls)).
  destruct (contig_app l0 (concat (l1 :: Ls)) h0 hc2 ltac:(lia))
    as [hc3 [hf3 hl3]].
  rewrite last_cons, <- hl2; split; [exact hc3|split; assumption].
Qed.

(** ** Coverage of the table *)

(** If the slices [l0 :: Ls] are each contiguous, join, and hold good
    lines, every significant from the first [M0] to the last [M0 + n]
    is in the range of a line that satisfies the six conditions. *)
Theorem cover_ok l0 Ls :
  Forall (fun l => contig l = true) (l0 :: Ls) ->
  Forall lines_good (l0 :: Ls) ->
  joins (l0 :: Ls) = true ->
  forall M, firstM l0 <= M <= lastM (last Ls l0) ->
  exists s M0 n B, In s (concat (l0 :: Ls)) /\
  parse s = Some (M0, n, B) /\ line_ok M0 n B /\ M0 <= M <= M0 + n.
Proof.
intros hc hg hj M hM.
destruct (contig_concat l0 Ls hc hj) as [hc1 [hf hl]].
rewrite <- hf, <- hl in hM.
destruct (contig_cover _ M hc1 hM) as [s [M0 [n [B [hs [hp hr]]]]]].
exists s, M0, n, B; split; [exact hs|split; [exact hp|split; [exact hr]]].
apply in_concat in hs as [l [hl0 hsl]].
rewrite Forall_forall in hg.
destruct (hg l hl0 s hsl) as [M1 [n1 [B1 [hp1 hok]]]].
rewrite hp in hp1; injection hp1 as -> -> ->; exact hok.
Qed.

```

G ExpParse.v

```

(** * Reading the lines of the table from their text

  A line of [in_lt] is kept verbatim as a primitive string:

      0x1.4678ea18f304cp+9 7412951889 0x6b4736c9ee3e2c16 ... 0x3be38432b106019d

  that is [x0] in hexadecimal, [n] in decimal, then [k * l] words in
  hexadecimal, each [B_i] being [l] words, the least significant first.
  [parse] turns such a line into [(M0, n, B)] with [x0 = M0 * 2^uexp]; a

```

```

line of another shape gives [None], and [check_raw] then answers
[false]. *)

```

```

From Stdlib Require Import ZArith List PrimString PrimStringAxioms Uint63.
From ExpTable Require Import ExpCheck.
Import ListNotations.

```

```

Open Scope Z_scope.

```

```

(** ** Characters *)

```

```

Definition c_space : int := 32%uint63.      (* ' ' *)
Definition c_0 : int := 48%uint63.          (* '0' *)
Definition c_9 : int := 57%uint63.          (* '9' *)
Definition c_a : int := 97%uint63.          (* 'a' *)
Definition c_f : int := 102%uint63.         (* 'f' *)
Definition c_x : int := 120%uint63.         (* 'x' *)
Definition c_dot : int := 46%uint63.        (* '.' *)
Definition c_p : int := 112%uint63.         (* 'p' *)
Definition c_plus : int := 43%uint63.       (* '+' *)
Definition c_l : int := 49%uint63.         (* 'l' *)

```

```

(** The value of a hexadecimal digit, or [None]. *)

```

```

Definition hexd (c : int) : option Z :=
  if (c_0 <=? c)%uint63 && (c <=? c_9)%uint63 then Some (to_Z (c - c_0))
  else if (c_a <=? c)%uint63 && (c <=? c_f)%uint63
  then Some (to_Z (c - c_a) + 10)
  else None.

```

```

(** The value of a decimal digit, or [None]. *)

```

```

Definition decd (c : int) : option Z :=
  if (c_0 <=? c)%uint63 && (c <=? c_9)%uint63 then Some (to_Z (c - c_0))
  else None.

```

```

(** ** Numbers *)

```

```

(** The number written with the digits [cs] in base [b], most
    significant first; [None] if a character is not a digit. *)

```

```

Fixpoint digits (d : int -> option Z) (b acc : Z) (cs : list int)
  : option Z :=
  match cs with
  | [] => Some acc
  | c :: cs' =>
    match d c with
    | Some v => digits d b (acc * b + v) cs'
    | None => None
    end
  end.

```

```

Definition hexv (cs : list int) : option Z :=
  match cs with [] => None | _ => digits hexd 16 0 cs end.

```

```

Definition decv (cs : list int) : option Z :=
  match cs with [] => None | _ => digits decd 10 0 cs end.

```

```

(** A word [0xhh...h]. *)

```

```

Definition wordv (cs : list int) : option Z :=
  match cs with
  | c0 :: cx :: cs' =>
    if (c0 =? c_0)%uint63 && (cx =? c_x)%uint63 then hexv cs' else None
  | _ => None
  end.

(** The characters before the first [p], and those after it. *)
Fixpoint split_p (cs : list int) : list int * list int :=
  match cs with
  | [] => ([], [])
  | c :: cs' =>
    if (c =? c_p)%uint63 then ([], cs')
    else let (a, b) := split_p cs' in (c :: a, b)
  end.

(** [x0 = 0x1.hhhhhhhhhhhhhp+E] with at most 13 digits after the point
    (trailing zeros may be left out) and [E = xbin]: its significand. *)
Definition x0v (cs : list int) : option Z :=
  match cs with
  | c0 :: cx :: c1 :: cd :: cs' =>
    if (c0 =? c_0)%uint63 && (cx =? c_x)%uint63 && (c1 =? c_1)%uint63 &&
      (cd =? c_dot)%uint63 then
      let (fr, ex) := split_p cs' in
      match fr, ex with
      | _ :: _, cplus :: ce =>
        if (cplus =? c_plus)%uint63 && Nat.leb (List.length fr) 13 then
          match digits hexd 16 1 fr, decv ce with
          | Some f, Some e =>
            if Z.eqb e xbin
            then Some (f * 16 ^ Z.of_nat (13 - List.length fr))
            else None
          | _, _ => None
          end
        else None
      | _, _ => None
      end
    else None
  | _, _ => None
  end
  end.

(** ** Lines *)

(** The fields of a line, separated by single spaces. *)
Fixpoint fields (cur : list int) (cs : list int) : list (list int) :=
  match cs with
  | [] => [rev cur]
  | c :: cs' =>
    if (c =? c_space)%uint63 then rev cur :: fields [] cs'
    else fields (c :: cur) cs'
  end.

(** [w_0 + w_1 beta + ... ], the words least significant first. *)
Definition wsum (ws : list Z) : Z := fold_right (fun w a => w + beta * a) 0 ws.

(** The [B_i], each of [l] words, from the list of all words. *)

```

```

Fixpoint group (c : nat) (ws : list Z) : list Z :=
  match c with
  | 0 => []
  | S c' => wsum (firstn (Z.to_nat l) ws) :: group c' (skipn (Z.to_nat l) ws)
  end.

(** All the words, or [None] if one is not a word. *)
Fixpoint wordsv (fs : list (list int)) : option (list Z) :=
  match fs with
  | [] => Some []
  | f :: fs' =>
    match wordv f, wordsv fs' with
    | Some w, Some ws => Some (w :: ws)
    | _, _ => None
    end
  end.

Definition parse (s : string) : option (Z * Z * list Z) :=
  match fields [] (to_list s) with
  | fx :: fn :: fw =>
    if Nat.eqb (List.length fw) (k * Z.to_nat l) then
      match x0v fx, decv fn, wordsv fw with
      | Some M0, Some n, Some ws => Some (M0, n, group k ws)
      | _, _, _ => None
      end
    else None
  | _ => None
  end.

(** ** The check on the text *)

Definition check_raw (ls : list string) : bool :=
  Z.ltb (2 * E) beta_l && Z.leb m lbits &&
  forallb (fun s => match parse s with
    | Some (M0, n, B) => check_line M0 n B
    | None => false
  end) ls.

```

H ExpParseSpec.v

```

(** * What a line of the table says, and [parse] reads exactly that

  A line of [in_lt] is the text

    0x1.F p+X N 0xW_0 0xW_1 ... 0xW_(k l - 1)

  with single spaces between the fields (none inside [0x1.Fp+X]):
  [F] is 1 to 13 hexadecimal digits, [X] decimal digits of value
  [xbin], [N] decimal digits and each [W_j] hexadecimal digits. So
  [x0 = (1 + F / 16^|F|) 2^xbin]; with
  [M0 = 16^13 + F 16^(13 - |F|)] this is [x0 = M0 2^-52 2^xbin], that
  is [x0 = M0 2^uexp] as [uexp = xbin - 52]. The line gives [n = N],
  and [B_i = w_(i l) + w_(i l + 1) beta + ... + w_(i l + l - 1)
  beta^(l-1)] for [i < k], [w_j] being the value of [W_j].

```

```

    [line_text] states this; [parse_sound] proves that whatever [parse]
    returns is what the text says. A character is its code, an [int]. *)

From Stdlib Require Import PrimString PrimStringAxioms Uint63.
From ExpTable Require Import ExpCheck ExpParse.
From Stdlib Require Import ZArith List Lia.
Import ListNotations.

Open Scope Z_scope.

(** ** Digits and numbers *)

(** The characters [c_0], [c_a], ... are those of [ExpParse]. *)

(** [c] is the decimal digit of value [d]: '0' .. '9'. *)
Definition decdigit (c : int) (d : Z) : Prop :=
  0 <= d <= 9 /\ to_Z c = to_Z c_0 + d.

(** [c] is the hexadecimal digit of value [d]: '0' .. '9', 'a' .. 'f'. *)
Definition hexdigit (c : int) (d : Z) : Prop :=
  decdigit c d /\ (10 <= d <= 15 /\ to_Z c = to_Z c_a + (d - 10)).

(** [d_0 b^(len-1) + d_1 b^(len-2) + ... + d_(len-1)], most significant
    first. *)
Fixpoint value (b : Z) (ds : list Z) : Z :=
  match ds with
  | [] => 0
  | d :: ds' => d * b ^ Z.of_nat (length ds') + value b ds'
  end.

(** [cs] is a nonempty decimal numeral of value [v]. *)
Definition dec_text (cs : list int) (v : Z) : Prop :=
  cs <> [] /\ exists ds, Forall2 decdigit cs ds /\ v = value 10 ds.

(** [cs] is a nonempty hexadecimal numeral of value [v]. *)
Definition hex_text (cs : list int) (v : Z) : Prop :=
  cs <> [] /\ exists ds, Forall2 hexdigit cs ds /\ v = value 16 ds.

(** [W] is a word [0xhh...h] of value [w]. *)
Definition word_text (W : list int) (w : Z) : Prop :=
  exists H, W = c_0 :: c_x :: H /\ hex_text H w.

(** [f 0 + f 1 + ... + f (N-1)]. *)
Fixpoint sumZ (f : nat -> Z) (N : nat) : Z :=
  match N with 0 => 0 | S N' => sumZ f N' + f N' end.

(** ** A line *)

Definition frac_digits : nat := 13. (* hex digits of a binary64 fraction *)

(** [cs] is the line [0x1.Fp+X N W_0 ... W_(k l - 1)] of [(M0, n, B)]. *)
Definition line_text (cs : list int) (M0 n : Z) (B : list Z) : Prop :=
  exists F f X N Ws ws,
  cs = [c_0; c_x; c_1; c_dot] ++ F ++ [c_p; c_plus] ++ X ++
    c_space :: N ++ concat (map (fun W => c_space :: W) Ws) /\
  (* x0 = 0x1.Fp+X, 1 to 13 digits after the point, X = xbin *)

```

```

hex_text F f /\ (length F <= frac_digits)%nat /\
M0 = 16 ^ Z.of_nat frac_digits +
      f * 16 ^ (Z.of_nat frac_digits - Z.of_nat (length F)) /\
dec_text X xbin /\
(* n = N *)
dec_text N n /\
(* the k l words w_j *)
length Ws = (k * Z.to_nat l)%nat /\ Forall2 word_text Ws ws /\
(* B_i = sum_(j < l) w_(i l + j) beta^j *)
B = map (fun i => sumZ (fun j => nth (i * Z.to_nat l + j) ws 0 *
                                   beta ^ Z.of_nat j) (Z.to_nat l))
      (seq 0 k).

(** ** Proofs *)

Lemma decdP c d : decd c = Some d -> decdigit c d.
Proof.
unfold decd, decdigit.
destruct ((c_0 <=? c)%uint63 && (c <=? c_9)%uint63) eqn:E1; [|discriminate].
intros [= <-]; apply andb_prop in E1 as [E1 E2].
apply leb_spec in E1, E2.
pose proof (to_Z_bounded c) as Hc.
change (to_Z c_0) with 48 in *; change (to_Z c_9) with 57 in *.
rewrite sub_spec; change (to_Z c_0) with 48.
rewrite Z.mod_small; lia.
Qed.

Lemma hexdP c d : hexd c = Some d -> hexdigit c d.
Proof.
unfold hexd, hexdigit.
destruct ((c_0 <=? c)%uint63 && (c <=? c_9)%uint63) eqn:E1.
{ intros H; left; apply decdP; unfold decd; rewrite E1; exact H. }
destruct ((c_a <=? c)%uint63 && (c <=? c_f)%uint63) eqn:E2; [|discriminate].
intros [= <-]; right; apply andb_prop in E2 as [E2 E3].
apply leb_spec in E2, E3.
pose proof (to_Z_bounded c) as Hc.
change (to_Z c_a) with 97 in *; change (to_Z c_f) with 102 in *.
rewrite sub_spec; change (to_Z c_a) with 97.
rewrite Z.mod_small; lia.
Qed.

Lemma digitsP d b acc cs v : digits d b acc cs = Some v ->
  exists ds, Forall2 (fun c x => d c = Some x) cs ds /\
    v = acc * b ^ Z.of_nat (length cs) + value b ds.
Proof.
revert acc; induction cs as [|c cs IH]; intros acc; cbn [digits].
- intros [= <-]; exists []; split; [constructor|simpl; ring].
- destruct (d c) as [x] eqn:Hc; [|discriminate].
  intros H; destruct (IH _ H) as [ds [H1 H2]].
  exists (x :: ds); split; [constructor; auto|].
  cbn [value List.length].
  rewrite <- (Forall2_length H1), H2, Nat2Z.inj_succ, Z.pow_succ_r by lia.
  ring.
Qed.

Lemma decvP cs v : decv cs = Some v -> dec_text cs v.

```

```

Proof.
unfold decv; destruct cs as [|c cs']; [discriminate|].
intros H; apply digitsP in H as [ds [H1 H2]].
split; [discriminate|]; exists ds; split.
- eapply Forall2_impl; [|exact H1]; intros ? ? ?; apply decdP; assumption.
- rewrite H2; ring.
Qed.

```

```

Lemma hexvP cs v : hexv cs = Some v -> hex_text cs v.
Proof.
unfold hexv; destruct cs as [|c cs']; [discriminate|].
intros H; apply digitsP in H as [ds [H1 H2]].
split; [discriminate|]; exists ds; split.
- eapply Forall2_impl; [|exact H1]; intros ? ? ?; apply hexdP; assumption.
- rewrite H2; ring.
Qed.

```

```

Lemma wordvP cs w : wordv cs = Some w -> word_text cs w.
Proof.
unfold wordv; destruct cs as [|c0 [|cx cs']]; try discriminate.
destruct (c0 =? c_0)%uint63 eqn:E0; [|discriminate|.
destruct (cx =? c_x)%uint63 eqn:Ex; [|discriminate|.
apply eqb_spec in E0, Ex; subst; intros H.
exists cs'; split; [reflexivity|apply hexvP; exact H].
Qed.

```

```

Lemma wordsvP fs ws : wordsv fs = Some ws -> Forall2 word_text fs ws.
Proof.
revert ws; induction fs as [|f fs IH]; intros ws; simpl.
- intros [= <-]; constructor.
- destruct (wordv f) as [w] eqn:Hf; [|discriminate|.
  destruct (wordsv fs) as [ws'] eqn:Hfs; [|discriminate|.
    intros [= <-]; constructor; [apply wordvP|apply IH]; auto.
Qed.

```

```

Lemma split_pP cs a b : split_p cs = (a, b) -> b <> [] ->
  cs = a ++ c_p :: b.
Proof.
revert a; induction cs as [|c cs IH]; intros a; simpl.
- intros [= <- <-]; contradiction.
- destruct (c =? c_p)%uint63 eqn:Ec.
  + apply eqb_spec in Ec; subst; intros [= <- <-]; reflexivity.
  + destruct (split_p cs) as [a' b'] eqn:Hs.
    intros [= <- <-] Hb; simpl; f_equal; apply IH; auto.
Qed.

```

```

Lemma x0vP cs M0 : x0v cs = Some M0 ->
  exists F f X,
  cs = [c_0; c_x; c_1; c_dot] ++ F ++ [c_p; c_plus] ++ X /\
  hex_text F f /\ (length F <= frac_digits)%nat /\
  M0 = 16 ^ Z.of_nat frac_digits +
    f * 16 ^ (Z.of_nat frac_digits - Z.of_nat (length F)) /\
  dec_text X xbin.

```

```

Proof.
unfold x0v; destruct cs as [|c0 [|cx [|c1 [|cd cs']]]]; try discriminate.
destruct (c0 =? c_0)%uint63 eqn:E0; [|discriminate|.

```

```

destruct (cx =? c_x)%uint63 eqn:Ex; [|discriminate|.
destruct (cl =? c_l)%uint63 eqn:E1; [|discriminate|.
destruct (cd =? c_dot)%uint63 eqn:Ed; [|discriminate|]; cbn [andb].
apply eqb_spec in E0, Ex, E1, Ed; subst.
destruct (split_p cs') as [fr ex] eqn:Hs.
destruct fr as [|c fr']; [discriminate|].
destruct ex as [|cplus ce]; [discriminate|].
destruct (cplus =? c_plus)%uint63 eqn:Ep; [|discriminate|.
destruct (Nat.leb_spec (length (c :: fr')) 13) as [H1|]; [|discriminate|.
cbn [andb].
destruct (digits hexd 16 1 (c :: fr')) as [f1|] eqn:Hd; [|discriminate|.
destruct (decv ce) as [e|] eqn:He; [|discriminate|.
destruct (Z.eqb_spec e xbin) as [->|]; [|discriminate|.
intros [= <-].
apply eqb_spec in Ep; subst.
apply split_pP in Hs; [|discriminate|.
apply digitsP in Hd as [ds [H1 H2]].
exists (c :: fr'), (value 16 ds), ce; split; [|split; [|split; [|split]]|.
- rewrite Hs; reflexivity.
- split; [discriminate|]; exists ds; split; [|reflexivity|.
  eapply Forall2_impl; [|exact H1|]; intros ? ? ?; apply hexdP; assumption.
- exact H1.
- change (f1 * 16 ^ Z.of_nat (13 - length (c :: fr')) =
      16 ^ Z.of_nat frac_digits + value 16 ds *
      16 ^ (Z.of_nat frac_digits - Z.of_nat (length (c :: fr')))).
  rewrite H2, Nat2Z.inj_sub by exact H1; unfold frac_digits.
  set (L := Z.of_nat (length (c :: fr'))).
  assert (H1 : 0 <= L <= 13) by (unfold L; lia).
  replace (16 ^ Z.of_nat 13) with (16 ^ L * 16 ^ (13 - L))
    by (rewrite <- Z.pow_add_r by lia; f_equal; lia).
  change (Z.of_nat 13) with 13; ring.
- apply decvP; exact He.
Qed.

Lemma fields_nil cur cs : fields cur cs <> [].
Proof.
revert cur; induction cs as [|c cs IH]; intros cur; simpl; [discriminate|].
destruct (c =? c_space)%uint63; [discriminate|apply IH].
Qed.

Lemma fieldsP cur cs f fs : fields cur cs = f :: fs ->
  rev cur ++ cs = f ++ concat (map (fun W => c_space :: W) fs).
Proof.
revert cur f fs; induction cs as [|c cs IH]; intros cur f fs; simpl.
- intros [= <- <-]; simpl; rewrite !app_nil_r; reflexivity.
- destruct (c =? c_space)%uint63 eqn:Ec.
  + apply eqb_spec in Ec; subst; intros [= <- <-].
    destruct (fields [] cs) as [|g gs] eqn:Hg; [now apply fields_nil in Hg|].
    apply IH in Hg; simpl in Hg; rewrite Hg; reflexivity.
  + intros H; apply IH in H; rewrite <- H; simpl.
    rewrite <- app_assoc; reflexivity.
Qed.

Lemma sumZ_ext f g N : (forall j, f j = g j) -> sumZ f N = sumZ g N.
Proof. intros H; induction N; simpl; [reflexivity|rewrite IHN, H; ring]. Qed.

```

```

Lemma sumZ_shift f N : sumZ f (S N) = f 0 + sumZ (fun j => f (S j)) N.
Proof.
induction N; [simpl; ring|].
change (sumZ f (S (S N))) with (sumZ f (S N) + f (S N)).
rewrite IHN; simpl; ring.
Qed.

Lemma sumZ_scale c f N :
  sumZ (fun j => c * f j) N = c * sumZ f N.
Proof. induction N; simpl; [ring|rewrite IHN; ring]. Qed.

Lemma wsum_firstn n ws : wsum (firstn n ws) =
  sumZ (fun j => nth j ws 0 * beta ^ Z.of_nat j) n.
Proof.
revert ws; induction n as [|n IH]; intros ws; [reflexivity|].
rewrite sumZ_shift; destruct ws as [|w ws].
- rewrite (sumZ_ext _ (fun j => 0 * beta ^ Z.of_nat (S j)))
  by (intros [|j]; reflexivity).
  rewrite sumZ_scale; unfold wsum; cbn [firstn nth fold_right]; ring.
- change (wsum (firstn (S n) (w :: ws))) with
  (w + beta * wsum (firstn n ws)).
  rewrite IH.
  rewrite (sumZ_ext (fun j => nth (S j) (w :: ws) 0 * beta ^ Z.of_nat (S j))
    (fun j => beta * (nth j ws 0 * beta ^ Z.of_nat j))).
  + rewrite (sumZ_scale beta (fun j => nth j ws 0 * beta ^ Z.of_nat j)).
    cbn [nth]; change (Z.of_nat 0) with 0; ring.
  + intros j; cbn [nth]; rewrite Nat2Z.inj_succ, Z.pow_succ_r by lia; ring.
Qed.

Lemma groupP c ws : group c ws =
  map (fun i => sumZ (fun j => nth (i * Z.to_nat l + j) ws 0 *
    beta ^ Z.of_nat j) (Z.to_nat l))
    (seq 0 c).
Proof.
revert ws; induction c as [|c IH]; intros ws; [reflexivity|].
change (group (S c) ws) with
  (wsum (firstn (Z.to_nat l) ws) :: group c (skipn (Z.to_nat l) ws)).
rewrite IH, wsum_firstn; cbn [seq map]; rewrite <- seq_shift, map_map.
f_equal; apply map_ext; intros i; apply sumZ_ext; intros j.
rewrite nth_skipn; f_equal; f_equal; lia.
Qed.

(** ** The theorem *)

(** Whatever [parse] returns is what the line says. *)
Theorem parse_sound s M0 n B :
  parse s = Some (M0, n, B) -> line_text (to_list s) M0 n B.
Proof.
unfold parse.
destruct (fields [] (to_list s)) as [|fx [|fn fw]] eqn:Hf; try discriminate.
destruct (Nat.eqb_spec (length fw) (k * Z.to_nat l)) as [Hl|];
  [|discriminate].
destruct (x0v fx) as [M] eqn:Hx; [|discriminate].
destruct (decv fn) as [n'] eqn:Hn; [|discriminate].
destruct (wordsv fw) as [ws] eqn:Hw; [|discriminate].
intros [= <- <- <-].

```

```

apply fieldsP in Hf; simpl in Hf.
apply x0vP in Hx as (F & f & X & -> & HF & HL & HM & HX).
exists F, f, X, fn, fw, ws.
split; [|split; [|split; [|split; [|split; [|split; [|split; [|split; [|split]]]]]]];
  auto.
- rewrite Hf; simpl; rewrite <- !app_assoc; reflexivity.
- apply decvP; exact Hn.
- apply wordsvP; exact Hw.
- exact (groupP k ws).
Qed.

(** ** A short example *)

(** Eight words [0x0]. *)
Definition zeros8 : string := " 0x0 0x0 0x0 0x0 0x0 0x0 0x0 0x0"%pstring.

(** [x0 = 0x1.8p+9 = 3 2^51 u], [n = 3], words [1, 2, 0, ..., 0]. *)
Definition example_line : string :=
  cat "0x1.8p+9 3 0x1 0x2 0x0 0x0 0x0 0x0 0x0 0x0"%pstring
  (cat zeros8 (cat zeros8 (cat zeros8 zeros8))).

Example parse_example : parse example_line =
  Some (3 * 2 ^ 51, 3, [1 + 2 * beta; 0; 0; 0; 0; 0; 0; 0]).
Proof. vm_compute; reflexivity. Qed.

```